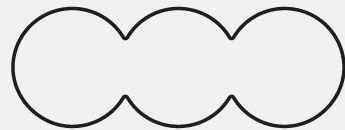
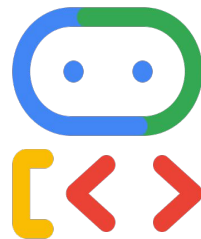
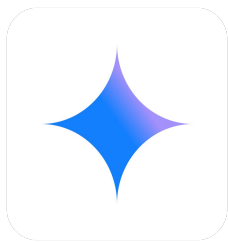




Google Developer Group

Build, Observe, Scale

Autonomous AI Agents with ADK, MCP Toolbox & Gemini Enterprise Agent Platform



{ Build  with AI }



Google Developer Group

Onkar Naik

Google Developer Expert for Google Cloud

DevOps Engineer @Callsign Technologies



Build  with AI

01 Introduction to Agentic AI & ADK



What is an Agentic AI?

Autonomous Action

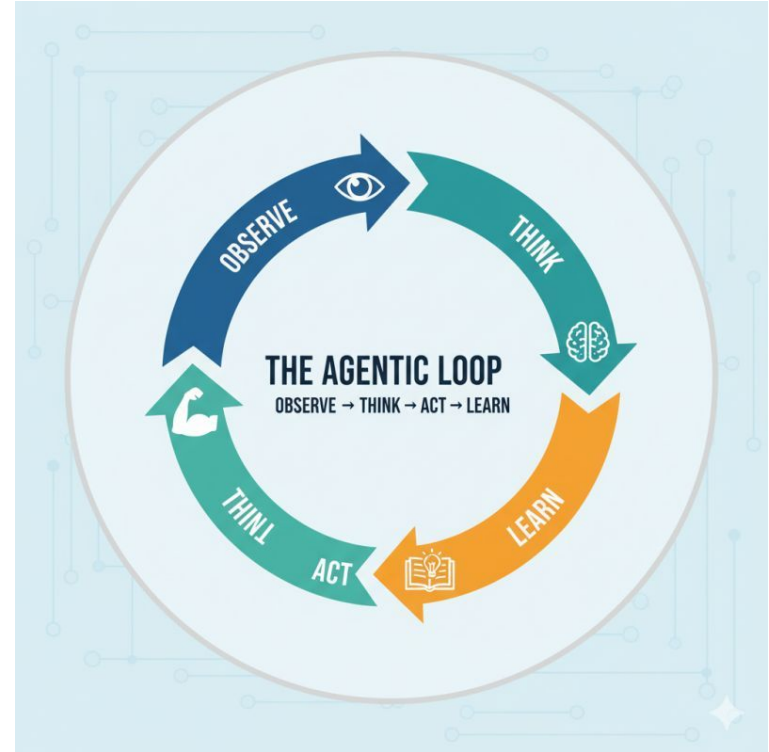
AI that sets goals, plans, and acts with minimal human input.

Cognitive Capability

Goes beyond prompts: it thinks, decides, orchestrates, and executes.

Reasoning Engine

Uses LLMs as reasoning engines paired with tools and real-world data.



Agent Development Kit (ADK)

A flexible and modular framework for **developing and deploying AI agents**.

Key Goals

- Make agent development feel like software development.
- Simplify creation, deployment, and orchestration.

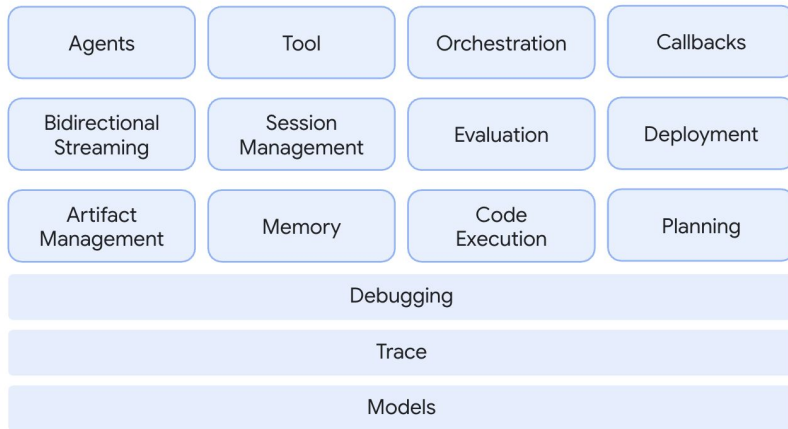
Core Principles

- **Model-agnostic:** Optimized for Gemini, supports others via LiteLLM.
- **Deployment-agnostic:** Local, Cloud Run, Agent Engine.
- **Compatibility:** Works with LangChain, CrewAI, etc.

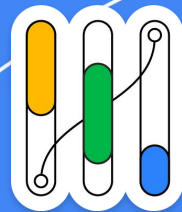
Discover, build and deploy agents

Agent Development Kit

Client side SDK to define multi-agent applications for complex, real world scenarios



02 Unlocking Data: An Introduction to MCP & Database Toolbox



Model Context Protocol (MCP)

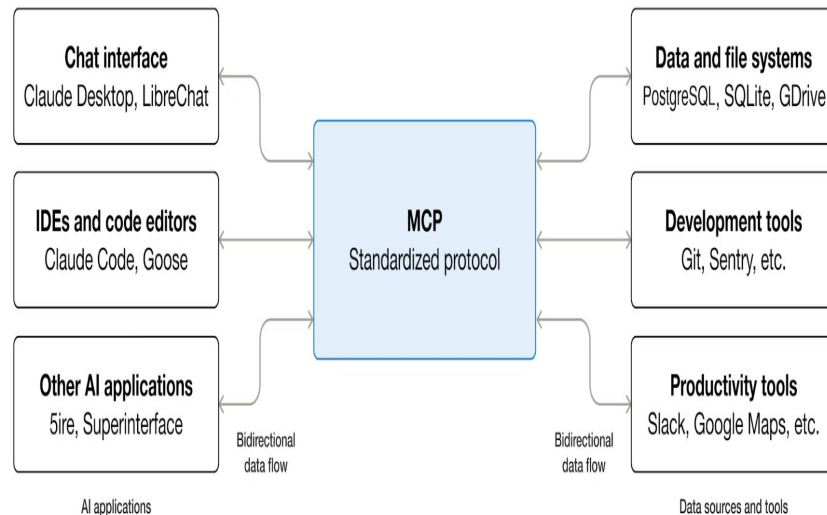
The Core MCP Concept: An open-source protocol that acts as a universal "USB port" for AI, establishing a standardized way for LLMs to safely connect to data and external tools.

Architecture & Cloud

- **Host vs. Server Decoupling:** Separates AI application from data source
- **Google-Managed MCP Servers:** secure, native access to GCP ecosystems with built-in IAM security

Building Blocks

- 📁 **Resources:** What the agent reads- DB schemas, files, issues etc
- 🛠️ **Tools:** What the agent does - query SQL, trigger APIs, update records.
- 📝 **Prompts:** What the agent knows how to ask - reusable templates for consistent tool invocation.



MCP Toolbox for Databases

Google's open source MCP server for simplified database interaction

Flexible Deployment Modes

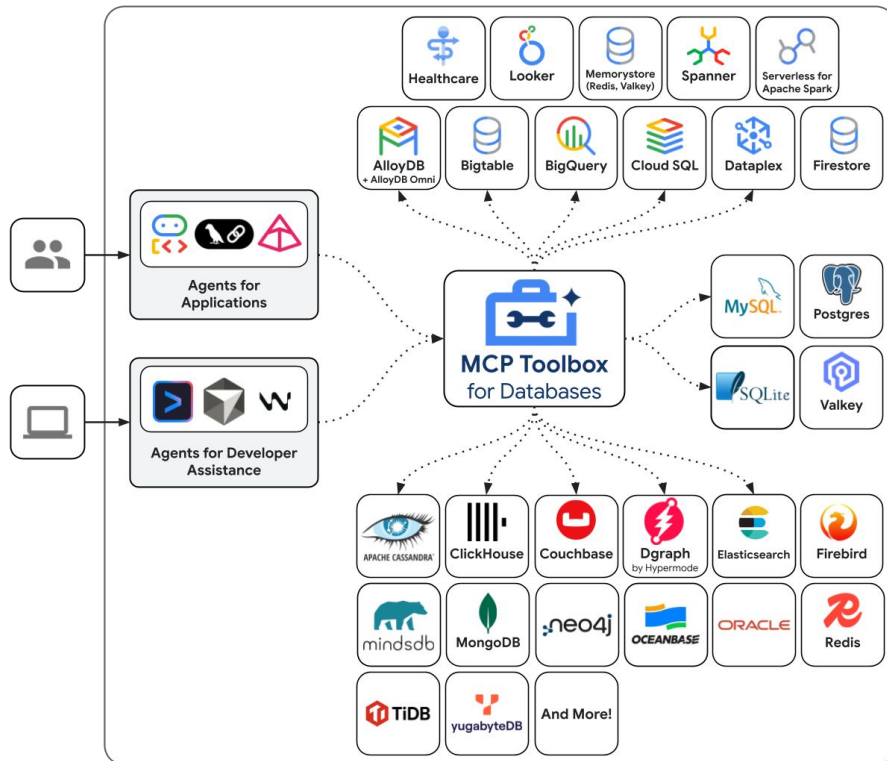
Build-Time: For rapid exploration. Plug Gemini CLI, Antigravity, or Claude Code to talk to DBs instantly.

Run-Time: For production-grade use. Define custom tools in `tools.yaml`.

Deploy Anywhere

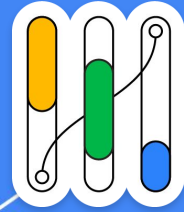
Cloud Run, GKE, Docker or local environments.

- Works with ADK, LangChain, and any MCP compatible client
- IAM auth + connection pooling In built
- Built-in Open-Telemetry & parameterized queries





03 The Gemini Enterprise Agent Platform



◆ Gemini Enterprise Agent Platform

Build

Agent Development Kit

New

3P agent frameworks

Agent Studio

New

Agent Garden

Gemini API and Model Garden

Gemini models

3P and open models

Model training

Model inference

Tools, data, and other agents

A2A

Grounding

RAG

MCP

Search

APIs and connectors

A2UI

AP2 and UCP

Cloud Marketplace

Scale

Agent Runtime

GA

Agent Sessions

GA

Agent Sandbox

GA

Agent Memory Bank

GA

Govern

Agent Gateway

New

Agent Identity

GA

Agent Registry

New

Agent Anomaly Detection

New

Model Armor

Agent Policy

Agent Security

New

Agent Compliance

Optimize

Agent Evaluation

New

Agent Simulation

New

Agent Observability

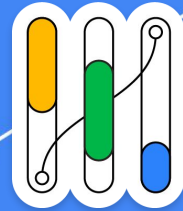
New

Agent Optimizer

New



04 Beyond Prompting Context Engineering



Context Engineering : Build Smarter Agents

Context Engineering

What you put in the context window determines what comes out — every time

- **Model Reality:** Prompting changes behavior; context engineering changes the model's reality
- **Pre-compute:** Give pre-computed answers instead of raw data to calculate
- **Efficiency:** Every unnecessary token delays the first word
- **Context discipline:** Relevant · Compressed · Structured
- **Context layers:** Persona · System prompt · Tools · Memory · Conversation history · User input

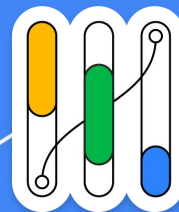
Context Caching

The infrastructure reward for optimized context

- **Static Layers:** System prompts and tool schemas are cached after the first request
- **Incremental Billing:** Subsequent turns primarily pay for new tokens instead of re-reading full history
- **Lower Marginal Cost:** Long multi-turn sessions become cheaper as repeated context is reused
- **Performance:** Agents get faster and cheaper over time, not more expensive
- **TTL-Based Storage:** Explicit caches persist temporarily (e.g. 300s) and incur small storage costs

Context engineering decides intelligence. Context caching decides economics

05 Demo



1. CLIENT & INVOCATION TIER



FinBot Web UI

- FastAPI backend
- SSE streaming to browser



Gemini CLI

- Local development
- Antigravity plugin

2. GEMINI ENTERPRISE AGENT PLATFORM



Agent Engine

- Orchestrator & Reasoning Loop
- AdkApp entry point
- Implicit session management via AdkApp

Orchestrates



Google Cloud ADK

- root_agent definition
- Gemini model
- Tier-based response formatting

Uses



Explicit Context Cache

- Configurable min tokens
- Configurable TTL
- 68% cache hit by Turn 7



Agent Identity

- Workload Identity Federation
- OIDC token authentication
- Least privilege IAM roles

Assumes Identity



Agent Service Account

- Workload Identity
- Least Privilege

Roles:

- cloudtrace.agent
- logging.logWriter
- monitoring.metricWriter
- bigquery.user

MCP Protocol

OIDC Token Auth

3. MODEL CONTEXT PROTOCOL (MCP) ECOSYSTEM



FinOps MCP Server (MCP Toolbox)

- MCP Toolbox for Databases
- Cloud Run
- Scale-to-zero (min=0)

Parameterized SQL



tools.yaml

- 9 pre-defined tools
- LIMIT 5
- Parameterized queries

4. BIGQUERY DATA WAREHOUSE



Billing Export Data

- Standard billing export
- Resource-level billing export
- All amounts in INR (₹)



ARIMA_PLUS ML Models

- Forecast ML Model
- ML_FORECAST()
- ML_DETECT_ANOMALIES()

5. END-TO-END OBSERVABILITY & TELEMETRY

Configured via 4 Otel env vars · Zero instrumentation code



Cloud Trace

- Distributed tracing
- Full span tree
- token counts



Cloud Logging

- Structured logs
- Trace-correlated
- Resource: reasoning_engine



Cloud Monitoring

- QPS · Latency · CPU
- Auto-instrumented
- Dashboards · SLOs



Otel Configuration (4 env vars)

- 1 GOOGLE_CLOUD_AGENT_ENGINE_ENABLE_TELEMETRY=true
- 2 OTEL_SEMCONV_STABILITY_OPT_IN=gen_ai_latest_experimental
- 3 OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT=EVENT_ONLY
- 4 ADK_CAPTURE_MESSAGE_CONTENT_IN_SPANS=false (PII protection)

How Observability + Context Caching Improved Agent Performance

Before Optimization

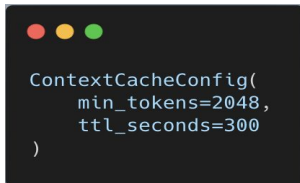
- Multi-turn sessions > 9,000 tokens
- Input processing ~6.5s
- Response synthesis up to 45s
- Repeated re-reading of prompt/tools
- Higher inference cost every turn

Latency

45s

What Changed

- Enabled Explicit Context Caching
- Cached system prompts & MCP schemas
- Observed reasoning bottlenecks
- Reduced repeated token processing

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It displays a Go struct configuration for context caching.

```
ContextCacheConfig(  
  min_tokens=2048,  
  ttl_seconds=300  
)
```

After Optimization

- Input: 6.5s → **0.8s**
- End-to-end: 45s → **11s**
- Faster multi-turn conversations
- Enterprise-grade scalability

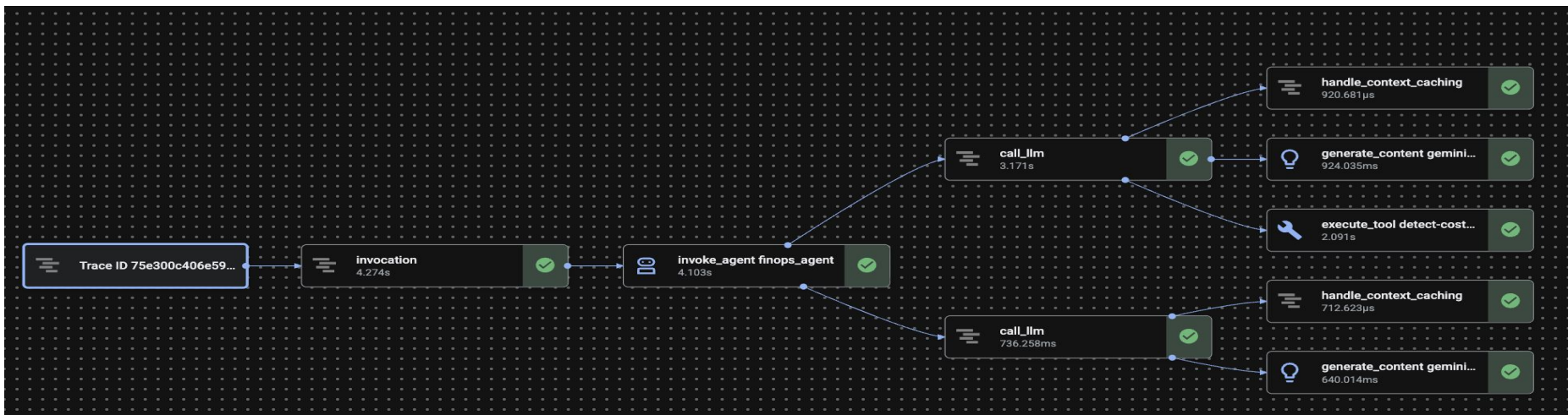
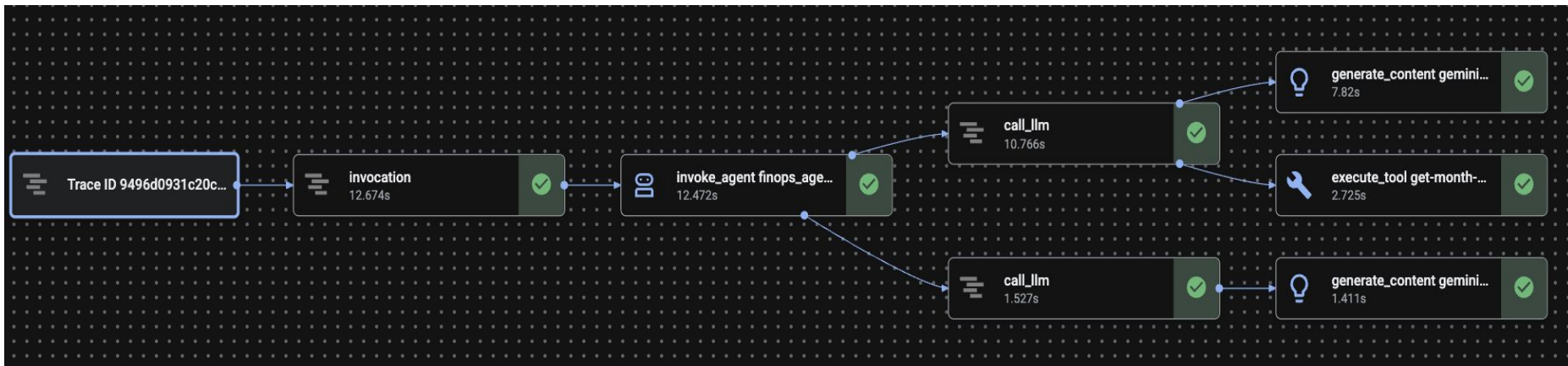
Latency

11s

Learnings

1. Tool execution wasn't the bottleneck — reasoning was.
2. Observability explained WHY agents slowed.
3. Context caching made long conversations cheaper.
4. Production AI needs grounding, telemetry, & identity.

Trace-Level Comparison: Context Caching



06 Q/A

